

SUBLISPORT GARCÍA

Informe completo de estructura del sistema

Documento técnico para auditoría, mejoras y trabajo con IA (Claude / Cursor)

Generado: 9 de junio de 2026 · Repositorio: github.com/ANTHONY109823/SubliSport-Garcia

Producción: sublisport-garcia-production.up.railway.app

Índice

1. Resumen ejecutivo
2. Stack tecnológico
3. Arquitectura en capas
4. Estructura de carpetas
5. Roles y permisos
6. Rutas del panel (Blazor)
7. API REST pública
8. Sitio público (landing)
9. Modelo de pedido y flujo
10. Estados del pedido
11. Motor de cotización web
12. Lista de jugadores (roster)
13. Base de datos
14. Servicios y helpers
15. Componentes Blazor compartidos
16. Despliegue (Railway / Docker)
17. Áreas de mejora sugeridas

1. Resumen ejecutivo

SubliSport García es un **sistema de gestión interno** para una empresa de confección deportiva sublimada. Combina:

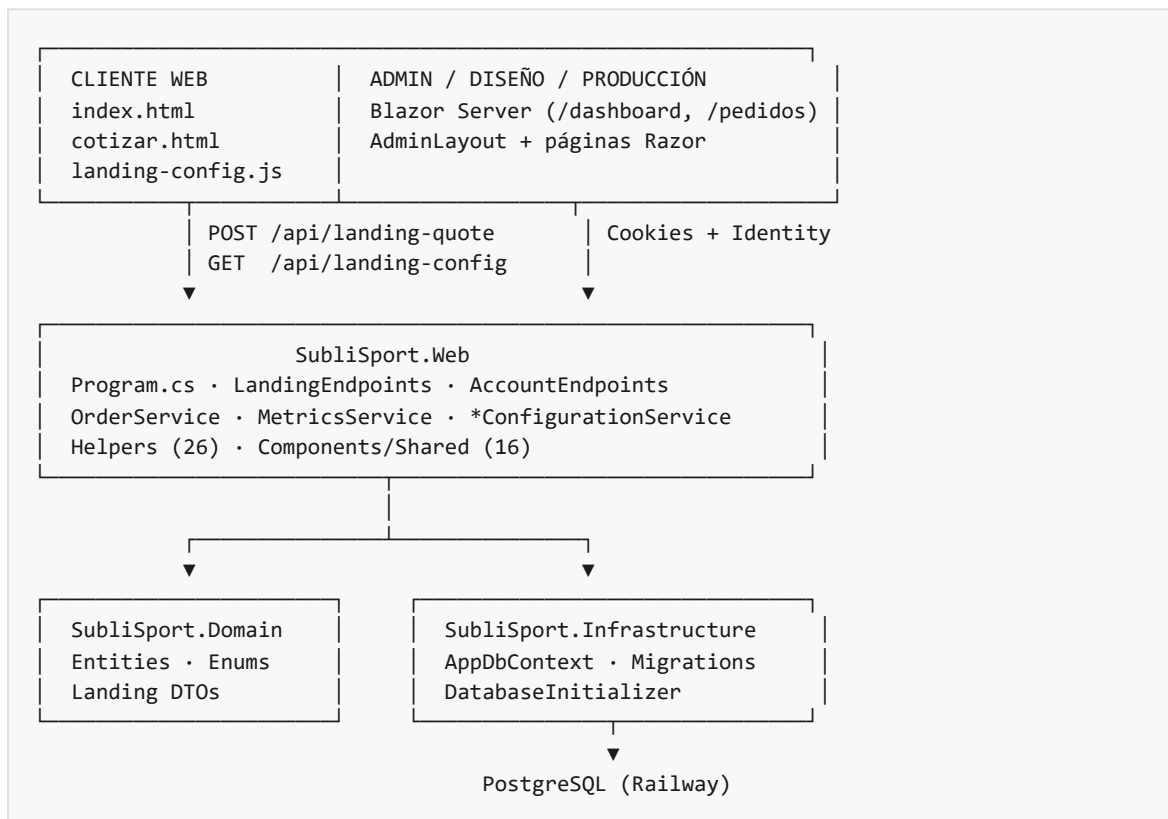
- **Panel administrativo** — Blazor Server (.NET 10) con login por roles.
- **Sitio público estático** — `index.html` (landing) y `cotizar.html` (formulario de cotización).
- **API REST** — Endpoints mínimos para cotizaciones web y configuración.
- **PostgreSQL** — Persistencia en Railway.

Usuarios principales: **Wilber (Admin)**, **diseñadores**, **Juanita (Producción)**, clientes vía web.

2. Stack tecnológico

Capa	Tecnología
Backend / UI admin	ASP.NET Core 10, Blazor Server (InteractiveServer)
Autenticación	ASP.NET Core Identity (cookies, roles)
ORM	Entity Framework Core + PostgreSQL (Npgsql)
Frontend público	HTML/CSS/JS vanilla (sin framework)
Configuración dinámica	JSON en tablas (Landing, Pricing, Production, PanelUi)
Contenedor	Docker (.NET 10 SDK build → aspnnet runtime)
Hosting	Railway (healthcheck /health, puerto PORT=8080)

3. Arquitectura en capas



4. Estructura de carpetas

Ruta	Contenido
/	Landing raíz: index.html, cotizar.html, landing-config.js, panel-config.js, assets/
src/SubliSport.Domain/	Entidades, enums, DTOs landing, modelos pricing (22 archivos)
src/SubliSport.Infrastructure/	DbContext, migraciones EF Core, seed DB

<code>src/SubliSport.Web/</code>	App Blazor, servicios, helpers, wwwroot (copia landing servida en prod)
<code>src/SubliSport.Web/Components/Pages/</code>	Páginas por rol: Dashboard, Orders, Production, Admin, Login
<code>src/SubliSport.Web/Components/Shared/</code>	Componentes reutilizables (roster, proforma, diseñador, etc.)
<code>src/SubliSport.Web/wwwroot/</code>	Estáticos servidos por Railway: cotizar.html, landing-config.js, admin.css, uploads/
<code>docs/</code>	Documentación (GUIA-RAILWAY.md, este informe)
<code>Dockerfile</code> , <code>railway.toml</code> , <code>start.sh</code>	Build y arranque en producción

Importante: Los archivos en la raíz (`cotizar.html` , `landing-config.js`) deben mantenerse sincronizados con `src/SubliSport.Web/wwwroot/` . Railway sirve desde `wwwroot`.

5. Roles y permisos

Rol	Constante	Acceso principal
SuperAdmin	AppRoles.SuperAdmin	Soporte: usuarios, config precios/WhatsApp/landing/panel. Modo impersonación.
Admin	AppRoles.Admin	Dashboard, pedidos, nuevo pedido, diseñadores, producción, asignación.
Designer	AppRoles.Designer	Mis diseños, aceptar/iniciar diseño, aprobación cliente.
Production	AppRoles.Production	Cola impresión/planchado/confección, historial, cobros.

Políticas en Program.cs: SuperAdminOnly, Administration, DesignTeam, ProductionTeam.

6. Rutas del panel (Blazor)

Ruta	Página	Roles
/login	Login.razor	Anónimo
/dashboard	Dashboard.razor	Todos (vista según rol)
/pedidos	OrderList.razor	Admin, Designer
/pedidos/nuevo	CreateOrder.razor	Admin
/pedidos/{id}	OrderDetail.razor	Admin, Designer
/pedidos/{id}/proforma	QuoteProformaPrint.razor	Admin
/pedidos/disenadores	DesignerStats.razor	Admin
/produccion	ProductionQueue.razor	Admin, Production
/produccion/{id}	ProductionOrderDetail.razor	Admin, Production
/produccion/historial	ProductionHistory.razor	Production
/produccion/{id}/ficha-confeccion	ConfectionPrintSheet.razor	Producción (PDF imprimible)
/usuarios	Users.razor	SuperAdmin
/configuracion/precios	PricingSettings.razor	SuperAdmin
/configuracion/whatsapp	WhatsAppSettings.razor	SuperAdmin
/configuracion/landing	LandingSettings.razor	SuperAdmin
/configuracion/panel	PanelUiSettings.razor	SuperAdmin

7. API REST y endpoints

Método	Ruta	Descripción
--------	------	-------------

GET	/health	Healthcheck Railway
GET	/	Redirect → /index.html
GET	/api/landing-config	Config cotizar (teléfonos, telas, prendas, CORS público)
POST	/api/landing-quote	Recibe cotización web → crea Order + guarda fotos
POST	/account/login	Login form POST
POST	/account/logout	Cerrar sesión
POST	/account/impersonate	SuperAdmin entra como otro usuario
POST	/produccion/pedidos/{id}/aceptar	Aceptar pedido en producción

8. Sitio público (landing)

index.html

Landing marketing: hero, servicios, galería, contacto. Tema oscuro dorado (#F0A500). Enlaza a cotizar.html y login del panel.

cotizar.html + landing-config.js

- Selector tipo cliente: Directo retail vs Servicio MYPE B2B.
- Tipo prenda: Conjunto, Camiseta, Mixto (chips).
- Tela, bordado (logo/marca/short círculos), cantidad, fecha entrega.
- **Lista jugadores:** nombre, talla, número, género, prenda (conjunto/camiseta).
- Import Excel/CSV, hasta 3 fotos referencia (base64 comprimido).
- Envío: POST a `/api/landing-quote` + opción WhatsApp con texto generado.

Config cargada desde `/api/landing-config` o fallback en DEFAULTS del JS.

9. Modelo de pedido (Order)

Entidad central en `SubliSport.Domain/Entities/Order.cs` . Campos clave:

Campo	Uso
OrderNumber	Identificador único visible
ClientName, ClientPhone, ClientEmail	Datos cliente
GarmentType, MixedGarmentDetails (JSON)	Prenda global o líneas mixtas
PricingTier	DirectRetail MypeB2B
Sport	Fijo "Fútbol" en flujo actual
Quantity, SizeRange	Cantidad y resumen tallas
ConfectionRosterDetails (JSON)	Lista jugadores: nombre, talla, número, gender, kitType
Status, Priority	Flujo operativo y urgencia
AssignedDesignerId	Diseñador asignado
FabricTypeName, FabricMeters, costos calculados	Producción y pricing
CalculatedTotal, ChargeAmount, PricingNotes	Cotización y proforma (incl. flag cotización web)
StatusHistory	Historial de cambios de estado

10. Estados del pedido (OrderStatus)

#	Estado	Fase
0	CotizacionRecibida	Entrada (web o manual)
1	EnRevision	Admin revisa
2	AsignadoDiseno	Con diseñador, pendiente aceptar
3	EnDiseno	Diseño activo
4	DisenoAprobado	Pasa a producción
5-7	EnImpresion / EnPlanchado / EnConfeccion	Producción
8	ListoEntrega	Por recoger
9	Entregado	Cerrado
10	Cancelado	Anulado

11. Motor de cotización web

Archivos: `LandingQuoteCalculator.cs` , `LandingQuoteOrderAssembler.cs` ,
`LandingQuoteSubmit.cs`

- Tablas precio conjunto/camiseta por tier de talla y tela (dry_fit, etc.).
- Bordados: logo, marca, short (unidades fijas por pieza).
- Con roster: precio **por fila** según kitType (conjunto vs camiseta).
- Genera: AdminSuggestionText, ClientProformaDraft, ClientRequestText (WhatsApp).
- OrderAssembler persiste pedido y serializa roster a ConfectionRosterDetails.

12. Lista de jugadores (roster)

Campo JSON	Valores	UI
name, size, number	Texto	Admin + cotizar + CSV
gender	Varon Femenino	PDF corte: VARON / CORTE PRINCESA
kitType	conjunto camiseta	Conjunto completo / Camiseta sola

Helpers: RosterGenderHelper, RosterKitHelper, RosterCsvImportHelper, ConfectionRosterHelper.

Plantilla CSV: `wwwroot/templates/plantilla-lista-sublisport.csv`

13. Base de datos (PostgreSQL)

Tabla	Descripción
Orders	Pedidos
OrderStatusHistories	Auditoría estados
AspNetUsers / Roles	Identity
PricingConfigurations	JSON tarifas calculadora admin
ProductionConfigurations	JSON reglas producción
LandingConfigurations	JSON config página cotizar
PanelUiConfigurations	JSON etiquetas UI panel

Migraciones en `Infrastructure/Data/Migrations/` (desde InitialCreate jun 2026).

14. Servicios (13) y Helpers (26)

Servicios principales

- **OrderService** — CRUD pedidos, transiciones estado, asignación diseñador, proforma landing.
- **MetricsService** — Dashboard admin/diseñador, leaderboard.
- **LandingConfigurationService** — Config JSON cotizar.
- **PricingConfigurationService / PricingCalculatorService** — Tarifas MYPE y calculadora metraje.
- **ProductionConfigurationService** — Reglas producción.
- **QuoteReferenceImageService** — Guarda fotos en wwwroot/uploads/quotes/.
- **UserManagementService / AuthService** — Usuarios e impersonación.

Helpers destacados

- DesignerOrderHelper, ProduccionOrderHelper — Lógica por rol.
- OrderJourneyHelper, OrderDisplayHelper, OrderStatusHelper — UI y timeline.
- LandingQuoteNotesHelper — Detecta cotizaciones web en PricingNotes.
- ProformaDocumentBuilder, OrderProformaBuilder — Documentos imprimibles.
- WhatsAppHelper — Links wa.me con mensajes.
- GarmentTypeHelper, MixedGarmentHelper — Prendas mixtas.

15. Componentes Blazor compartidos

Componente	Función
AdminQuoteRosterEditor	Lista jugadores admin + import CSV
AdminQuotePhotoSlots	Fotos referencia en registrar pedido
DesignerPickerCards	Selector diseñador con métricas carga
ClientTypeSelector / ServiceTypeSelector	Directo vs MYPE
OrderPricingCalculator	Calculadora metraje producción
ProductionOptionsPanel	Lista confeccionista en detalle producción
OrderTimeline	Historial visual del pedido
ProformaInvoice	Vista proforma cliente
StatRing / DonutChart	Métricas dashboard

16. Despliegue

- **GitHub:** ANTHONY109823/SubliSport-Garcia (rama main).
- **Railway:** Auto-deploy desde main, Dockerfile multi-stage, start.sh ejecuta migraciones.
- **Variables:** DATABASE_URL (PostgreSQL), PORT, ASPNETCORE_ENVIRONMENT.
- **Estáticos:** wwwroot servido por Kestrel; landing accesible en /index.html y /cotizar.html.

- **CORS LandingPublic:** github.io, railway.app, localhost.

17. Áreas de mejora sugeridas (para Claude/Cursor)

Área	Observación	Prioridad
Duplicación landing	Raíz y wwwroot deben sincronizarse manualmente	Alta
Tests	Sin tests automatizados visibles	Alta
API documentada	Sin OpenAPI/Swagger	Media
Blazor Server	Escalabilidad limitada (SignalR); considerar WASM o híbrido a futuro	Media
Uploads	Fotos en disco local; en Railway puede perderse al redeploy	Alta
PricingNotes	Campo multipropósito (proforma + flag web); podría normalizarse	Media
Selects nativos cotizar	Dependen de color-scheme del OS; considerar custom dropdown	Baja
Notificaciones	Sin email/push al recibir cotización web	Media
Mobile admin	Tablas anchas; mejorar responsive	Media
i18n	Solo español; hardcoded strings	Baja

18. Flujo típico end-to-end

Cliente → cotizar.html → POST /api/landing-quote → LandingQuoteCalculator + OrderService.CreateManualOrder → Order (CotizacionRecibida) + fotos en uploads/
Admin → /pedidos/nuevo o revisa web → asigna diseñador → Designer acepta → EnDiseno → aprobación cliente → DisenoAprobado → Producción (impresión/planchado/confección) → ListoEntrega → Entregado
PDF confección: /produccion/{id}/ficha-confeccion (roster + corte + prenda)